



Software para Espectrofotómetro AS726X

Manual de Arquitectura de Software

Sebastian H. Betancur
Grupo de Investigación Biomicrosystems
Universidad de los Andes

Agosto de 2025

Índice general

1	Visión General de la Arquitectura	2
1.1	Propósito del Documento	2
1.2	Filosofía de Diseño y Patrones	2
1.3	Jerarquía e Interacción entre Clases	3
1.4	Decisiones Clave y Alternativas Consideradas	4
2	Guía de Despliegue y Dependencias	5
2.1	Dependencias de Software	5
2.2	Instalación del Entorno	6
3	El Controlador: Lógica y Comunicación	7
3.1	La Aplicación Principal: <code>main.py</code>	7
3.2	Controladores de Lógica > <code>filemanager.py</code> , <code>calculationcontroller.py</code>	12
3.3	Comunicación Asíncrona: <code>workers.py</code>	14
3.4	El Firmware: <code>Espectrofotometro.ino</code>	14
4	El Modelo: Gestión del Estado y los Datos	16
4.1	Estructuras de Estado Centralizadas: <code>app_state.py</code>	16
5	La Vista: Interfaz de Usuario	18
5.1	El Panel Base: <code>base_panel.py</code>	18
5.2	Paneles de Pestañas (Implementaciones de <code>BasePanel</code>)	19
5.3	Diálogos y Widgets de Utilidad	21
6	Módulos de Soporte y Constantes	23
6.1	Constantes Globales: <code>constants.py</code>	23
6.2	Tipos y Enumeraciones: <code>measurement_type.py</code>	24

Visión General de la Arquitectura

1.1 Propósito del Documento

Este manual detalla la arquitectura interna del software de control desarrollado para el espectrofotómetro AS726X. Su objetivo es servir como una guía técnica exhaustiva para desarrolladores, con el fin de facilitar el mantenimiento, la depuración y la extensión de funcionalidades sobre la base de código existente. El documento asume que el lector posee conocimientos básicos de programación orientada a objetos y de Python.

1.2 Filosofía de Diseño y Patrones

La arquitectura del software se fundamenta en un conjunto de principios de diseño orientados a garantizar la robustez, mantenibilidad y escalabilidad del sistema.

La estructura principal se inspira en el conocido patrón **Modelo-Vista-Controlador (MVC)**, que promueve una clara separación de responsabilidades. El **Modelo**, representado por las dataclasses en `app_state.py`, funciona como un contenedor pasivo para los datos y el estado de la aplicación. La **Vista**, compuesta por los componentes de la interfaz gráfica de PyQt5, se encarga exclusivamente de la presentación de la información y de la captura de las interacciones del usuario. En el centro, el **Controlador**, implementado en la clase principal `SpectrometerApp`, actúa como el intermediario que orquesta la comunicación entre la Vista y el Modelo, y ejecuta la lógica de la aplicación.

Este enfoque se complementa con un **diseño altamente modular**, donde cada funcionalidad importante está encapsulada en su propio módulo. La comunicación entre estos módulos se gestiona a través de un paradigma de **programación orientada a eventos**, utilizando el mecanismo de señales y slots de Qt. Esto crea un sistema desacoplado, donde los componentes emiten "señales" para notificar eventos, y otros componentes se "suscriben" a ellas a través de "slots", sin necesidad de dependencias directas.

Finalmente, para garantizar que la interfaz de usuario nunca se bloquee, las operaciones potencialmente lentas como la comunicación con el hardware se manejan de forma **asíncrona**. La lectura del puerto serie se delega a un hilo de ejecución secundario, que trabaja en segundo plano y notifica al hilo principal mediante señales cuando llegan nuevos datos, asegurando una experiencia de usuario siempre fluida.

1.3 Jerarquía e Interacción entre Clases

El componente central, que actúa como el orquestador de la aplicación, es la clase **SpectrometerApp**. Esta clase, que define la ventana principal, contiene las instancias de los demás componentes importantes: los paneles de la interfaz, los controladores de lógica como **FileManager** y **CalculationController**, y los objetos de estado del Modelo. Es el único componente con una visión global del sistema y la responsabilidad de coordinar todas las operaciones.

La interfaz de usuario (la Vista) está organizada bajo una jerarquía de herencia. Se definió una clase base, **BasePanel**, que actúa como una plantilla abstracta para todas las pestañas principales. Esta clase base establece una estructura visual común y una interfaz de comunicación estándar. Los paneles específicos como **MainSpectrumPanel** o **CalibrationCurveTab** heredan de **BasePanel** para implementar su funcionalidad particular. De esta forma se busca mantener la consistencia visual y funcional en toda la aplicación y la reutilización de código.

La interacción entre estos componentes se gestiona a través del **mecanismo de señales y slots de Qt**. En lugar de llamadas directas a métodos, los componentes se comunican de forma indirecta. Por ejemplo, cuando un usuario presiona un botón en un panel (Vista), este emite una señal (ej. `requestMeasurement`). La clase **SpectrometerApp** (Controlador) conecta previamente esa señal a uno de sus propios métodos (un slot), que ejecuta la lógica de medición. Este mismo mecanismo permite la comunicación segura entre hilos: el **SerialWorker**, desde su hilo secundario, emite la señal `data_received`, que es gestionada por un slot en **SpectrometerApp** en el hilo principal de la interfaz.

1.4 Decisiones Clave y Alternativas Consideradas

La selección de tecnologías para el proyecto se basó en una evaluación de alternativas y sus respectivos compromisos (*trade-offs*).

Se eligió **PyQt5** como framework para la interfaz de usuario sobre otras alternativas por su robusto conjunto de widgets y su excelente integración con Matplotlib, un requisito fundamental para la visualización de gráficos científicos.

Para el manejo de la concurrencia, se optó por el sistema de hilos nativo de Qt, **QThread**. Esta decisión se tomó por la integración nativa y sin fisuras con el bucle de eventos de PyQt5, ya que el uso de señales y slots entre hilos simplifica la arquitectura y reduce errores.

Finalmente, la estructura de la interfaz se diseñó de forma **modular por paneles**, donde cada pestaña es una clase independiente. Esto, si bien requiere una clase base y un sistema de señales, se alinea con el principio de Responsabilidad Única y facilita enormemente el mantenimiento y la escalabilidad del software a largo plazo.

Guía de Despliegue y Dependencias

Esta sección detalla los requisitos de software y los pasos necesarios para configurar el entorno de desarrollo y ejecutar la aplicación.

2.1 Dependencias de Software

La aplicación está construida sobre Python y requiere varias librerías externas para funcionar.

- **Python:** Se recomienda la versión 3.8 o superior.
- **PyQt5:** Es el framework principal para la construcción de la interfaz gráfica de usuario.
- **Matplotlib:** Utilizada para la creación de todos los gráficos y visualizaciones de datos.
- **NumPy:** Es la librería fundamental para la computación numérica y el manejo de datos espectrales.
- **SciPy:** Utilizada específicamente para su módulo de estadísticas para calcular las curvas de calibración.
- **pyserial:** Proporciona la funcionalidad para la comunicación con el puerto serie del microcontrolador.

- **Arduino IDE:** Necesario para compilar y cargar el firmware (`Espectrofotometro.ino`) en la placa.

2.2 Instalación del Entorno

La forma recomendada de gestionar las dependencias es a través de un entorno virtual y el gestor de paquetes `pip`.

1. **Crear un entorno virtual (Recomendado):**

```
python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate
```

2. **Crear un archivo `requirements.txt`:** Cree un archivo de texto con este nombre en la raíz del proyecto y añada el siguiente contenido o use el archivo adjunto:

```
PyQt5==5.15.10
matplotlib==3.8.4
numpy==1.26.4
scipy==1.13.1
pyserial==3.5
```

3. **Instalar las dependencias:** Con el entorno virtual activado, ejecute el siguiente comando en la terminal:

```
pip install -r requirements.txt
```

4. **Ejecutar la aplicación:** Una vez instaladas las dependencias, la aplicación se puede iniciar ejecutando el script:

```
python main.py
```

El Controlador: Lógica y Comunicación

Esta capa es el cerebro de la aplicación, responsable de la lógica de negocio, la orquestación de la UI y la comunicación con el mundo exterior.

3.1 La Aplicación Principal: `main.py`



Clase `SpectrometerApp(QMainWindow)`

Esta clase es el orquestador central. Hereda de `QMainWindow` de `PyQt5` y es responsable de inicializar y conectar todos los componentes, gestionar el estado global de la aplicación y manejar todos los eventos principales, ya sean de la UI o del hardware.

`__init__()`

Constructor de la clase. Inicializa la ventana principal, crea las instancias de las clases de estado, los timers, los controladores de lógica, y llama a los métodos internos para construir la UI y conectar las señales.

`_set_theme(is_dark)`

Aplica la hoja de estilos (QSS) del tema claro u oscuro a toda la aplicación y notifica a los paneles para que actualicen sus gráficos.

`_manual_toggle_theme(checked)`

Se activa cuando el usuario cambia el tema desde el menú y llama a


```
    _set_theme.  
_apply_initial_theme()  
    Se llama una sola vez al inicio para aplicar el tema por defecto a la  
    aplicación.  
_create_menu()  
    Construye y configura la barra de menú superior de la aplicación.  
_create_widgets()  
    Crea todas las instancias de los widgets de la interfaz en memoria.  
_update_logo()  
    Carga y reescala el logo correspondiente al tema actual para mostrarlo en  
    la UI.  
resizeEvent(event)  
    Se activa al cambiar el tamaño de la ventana para reescalar el logo dinámi-  
    camente.  
showEvent(event)  
    Se activa la primera vez que la ventana se muestra, para cargar el logo y  
    el mensaje de bienvenida.  
_show_startup_info()  
    Muestra un diálogo emergente al inicio para advertir al usuario sobre el  
    cierre seguro de la aplicación.  
_create_layouts()  
    Ensambla todos los widgets creados en la disposición final de la ventana.  
_connect_signals()  
    Conecta todas las señales emitidas por los widgets y paneles a sus respecti-  
    vos slots en esta clase, centralizando el manejo de eventos.  
_on_plot_type_changed(plot_type_enum)  
    Se activa al cambiar el tipo de gráfico (Intensidad, Absorbancia, etc.) y  
    redibuja el gráfico.  
_on_superimpose_selection_changed(labels)  
    Se activa cuando el usuario cambia la selección de espectros a superponer  
    y actualiza el gráfico.  
_on_log_scale_y_toggled(checked)  
    Cambia la escala del eje Y del gráfico principal entre lineal y logarítmica.  
_on_show_markers_toggled(checked)  
    Muestra u oculta los marcadores de puntos en las líneas del gráfico.  
_on_show_I0_toggled(checked)  
    Permite visualizar la línea de referencia  $I_0$  en el gráfico de intensidad.  
_on_calibration_applied(params)  
    Recibe los parámetros de una nueva curva de calibración desde el panel de
```

calibración y los guarda en el estado.

`_on_transfer_model_applied(params)`
Recibe los parámetros de un nuevo modelo de transferencia y los guarda en el estado.

`_on_concentration_config_changed()`
Gestiona las dependencias lógicas entre los controles de cálculo de concentración.

`_populate_ports(called_by_timer)`
Busca y añade los puertos seriales disponibles al menú desplegable.

`_check_for_new_ports_and_autoconnect()`
Tarea periódica que revisa si hay nuevos dispositivos seriales y trata de autoconectar.

`_connect_serial()`
Gestiona la lógica de conexión: abre el puerto serial e inicia el SerialWorker en un hilo secundario.

`_disconnect_serial(due_to_error)`
Inicia el proceso de desconexión segura del puerto serial.

`_perform_actual_disconnect(due_to_error)`
Realiza la desconexión efectiva: detiene el hilo, cierra el puerto y actualiza la UI.

`_handle_serial_error_message(error_message)`
Recibe y registra en el log los errores de comunicación reportados por el SerialWorker.

`_handle_connection_lost()`
Se activa cuando la conexión con el dispositivo se pierde y fuerza una desconexión limpia.

`_on_worker_finished()`
Se activa cuando el hilo del worker termina su ejecución para una limpieza de recursos.

`_send_command_direct(command)`
Envía un comando al puerto serial sin validaciones ni registro en log.

`_send_command(command)`
Es el método principal para enviar comandos al hardware, validando la conexión y registrando la acción.

`_handle_serial_data(raw_line)`
Es el manejador central de datos entrantes. Procesa cada línea recibida del Arduino y la enruta al método de procesamiento adecuado.

```
_process_error_msg(parts,  
raw_line)  
    Procesa un mensaje de error explícito enviado por el dispositivo.  
_process_calblanco_data(parts,  
raw_line)  
    Procesa y almacena los datos de la medición de referencia ( $I_0$ ).  
_process_blanco_data(parts,  
raw_line)  
    Procesa un espectro de medición de muestra ( $I$ ).  
_process_medicion_detenida_msg(parts,  
raw_line)  
    Procesa el mensaje de confirmación de detención enviado por el Arduino.  
_process_direct_measurement_data(command_type,  
...)  
    Maneja los datos recibidos durante una medición directa solicitada por un  
    panel secundario.  
_check_preconditions_for_measurement(name)  
    Valida que se cumplan las condiciones para iniciar una medición.  
_measure_reference_i0()  
    Inicia el flujo de trabajo para medir la referencia (blanco).  
_toggle_measurement_config_widgets()  
    Habilita o deshabilita los inputs de configuración según el modo de medición.  
_start_unified_measurement()  
    Inicia una de las mediciones principales (Continua, Única o Secuencial).  
_stop_current_measurement(silent)  
    Detiene la medición principal que esté en curso.  
_finish_current_measurement(success,  
...)  
    Finaliza una medición, enviando el comando "detener" restaurando el estado  
    de la aplicación.  
_process_collected_sub_samples()  
    En los modos Único o Secuencial, promedia las sub-muestras acumuladas.  
request_single_value_measurement(panel_id,  
...)  
    Atiende las solicitudes de mediciones directas provenientes de otros paneles.  
_handle_direct_measurement_timeout(reason)  
    Manejador para cuando una medición directa falla.  
_full_plot_update()  
    Recopila todo el estado y solicita un redibujado completo al panel principal.  
_on_calculate_and_register_concentration_clicked()  
    Inicia el cálculo de concentración y registra el resultado.
```

```
_on_IO_blank_subtraction_toggled(state)
    Activa o desactiva la sustracción de blanco.
set_active_calibration_curve(slope,
...)
    Guarda en el estado una nueva curva de calibración de sesión.
set_active_transfer_model(slope_t,
...)
    Guarda en el estado un nuevo modelo de transferencia.
clear_active_transfer_model(wavelength_nm)
    Elimina un modelo de transferencia activo para una longitud de onda.
_save_reference_calibration_params()
    Guarda en el estado los parámetros de una calibración de referencia externa.
_load_reference_calibration_params_for_wl()
    Carga en la UI los parámetros de calibración de referencia guardados.
_toggle_ref_cal_inputs()
    Habilita los inputs para 'curva' o 'factor' según el tipo de calibración de
    referencia.
_update_ref_cal_display(wavelength_nm)
    Actualiza la etiqueta de la UI que resume los parámetros de referencia
    guardados.
_update_status_label(text,
style)
    Actualiza el texto y color de la etiqueta de estado de conexión.
_update_status_label_style()
    Refresca el estilo de la etiqueta de estado al cambiar de tema.
_update_value_labels(value_array,
...)
    Función auxiliar para actualizar las etiquetas de la UI con los valores de un
    espectro.
_update_numeric_displays()
    Actualiza los displays numéricos de  $I_0$  e  $I$ .
_update_ui_state()
    Habilita o deshabilita todos los controles de la interfaz según el estado
    actual de la aplicación.
_log(message, is_error,
is_debug)
    Añade un mensaje con timestamp al log de eventos.
_show_error(message)
    Muestra un diálogo de error crítico al usuario.
```

```
_show_info(message)
    Muestra un diálogo de información general.
_show_help()
    Muestra la ventana principal de ayuda.
_show_context_help(help_id_key)
    Muestra una ayuda contextual específica.
_show_about_dialog()
    Muestra el diálogo 'Acerca de...'.
closeEvent(event)
    Se activa al cerrar la ventana para asegurar un apagado seguro.
_finalize_close(event)
    Realiza los pasos finales de cierre, como desconectar el puerto serial.
_update_superimpose_list()
    Actualiza la lista de mediciones disponibles para superponer.
_save_session_data()
    Invoca al FileManager para guardar los datos de la sesión.
_clear_session_data_internal(log)
    Reinicia todos los objetos de estado, limpiando la memoria.
_clear_session_data()
    Pide confirmación al usuario antes de borrar todos los datos de la sesión.
```

3.2 Controladores de Lógica > filemanager.py , calculationcontroller.py



Clase FileManager

Para mantener el código principal limpio y enfocado en la lógica de la aplicación, todas las operaciones de archivos se encapsularon en esta clase dedicada. Su única responsabilidad es manejar la lectura y escritura de datos en el disco.

```
__init__(app)
    Guarda una referencia a la aplicación principal para poder acceder a los
    datos que necesita guardar.
export_log()
    Abre un diálogo de "Guardar como..." y escribe el contenido del log de
    eventos a un archivo de texto.
export_graph_active_tab()
    Guarda una imagen de alta resolución del gráfico de la pestaña visible.
```

save_session_data()
Guarda todos los datos de la sesión (referencia y mediciones) en un archivo CSV.

save_selected_superimposed_data()
Guarda en un CSV únicamente los espectros que el usuario ha seleccionado en la lista de superposición.

get_save_filename_for_sequential()
Pide al usuario la ubicación para guardar los resultados de una medición secuencial antes de que comience.

save_sequential_data(m_state)
Escribe los resultados de una medición secuencial a un archivo CSV.

_get_active_figure_and_prefix()
Método interno que determina qué pestaña está activa para saber qué gráfico exportar.

_get_save_filename(title, ...)
Abre un diálogo de "Guardar como...." estándar, sugiriendo un nombre de archivo con timestamp.

_write_csv(filename, data_lines, ...)
Función interna que maneja la escritura de datos a un archivo CSV de manera formateada.



Clase CalculationController

Esta clase encapsula toda la lógica matemática para el cálculo de concentración. Fue aislada para que el código de la clase principal fuera más legible y para tener toda la matemática en un solo lugar.

__init__(app)
Guarda una referencia a la aplicación principal para acceder a los datos del espectro y a los parámetros de calibración activos.

update_display_only()
Calcula la concentración con los datos actuales y actualiza las etiquetas de la UI en tiempo real, sin registrar el dato.

calculate_and_register()
Calcula la concentración y guarda el espectro actual como una nueva medición en la lista de la sesión.

_calculate_concentration_core()
Es la función central de la clase. Implementa la lógica para decidir qué método de cálculo usar (Modelo de Transferencia, Curva de Sesión o Ley de Beer-Lambert) y aplica la matemática correspondiente.

`_get_current_absorbance_spectrum()`

Calcula el espectro de absorbancia actual ($A = -\log_{10}(I/I_0)$) a partir de la intensidad de la muestra y de la referencia.

3.3 Comunicación Asíncrona: `workers.py`

Este módulo es crucial para que la aplicación sea fluida. Define el "trabajador" que se ejecuta en un hilo separado para manejar la comunicación serial.

Clase `SerialWorker(QObject)`

Su única misión es leer continuamente el puerto serie sin bloquear la interfaz. Cuando recibe un dato, emite una señal para notificar al hilo principal, manteniendo la UI siempre responsiva.

`__init__(serial_instance)`

Recibe la instancia del puerto serial que fue abierta en la clase principal.

`run()`

Es el bucle principal que se ejecuta en el hilo secundario. Mientras esté activo, lee los datos del puerto y emite la señal `data_received` con cada nueva línea.

`stop()`

Se llama desde el hilo principal para detener al worker de forma segura, finalizando el bucle de lectura.

3.4 El Firmware: `Espectrofotometro.ino`

Este es el código que se ejecuta en el microcontrolador Arduino. Su función es ser el puente de bajo nivel entre el software y el hardware físico (sensor y LED). Fue diseñado para ser simple y reactivo.

`setup()`

Se ejecuta una vez al iniciar el Arduino. Configura la comunicación serial, inicializa el sensor y prepara el pin del LED.

`loop()`

Es el corazón del firmware, se ejecuta continuamente. Revisa si ha llegado un comando del software y, si está en modo de medición, toma una lectura y la envía.

`processCommand(cmd)`

Parsea los comandos que llegan por serial (calibrar", .empezar", "detener") y cambia el estado del Arduino.

`printMeasurements(prefix)`

Toma la lectura de los 6 canales del sensor, la formatea como una cadena de texto y la envía por el puerto serie al software.

`setWhiteLEDOn() /`

`setWhiteLEDOff()`

Funciones simples para encender y apagar el LED.

El Modelo: Gestión del Estado y los Datos

Esta capa define la estructura de todos los datos de la aplicación. Para esto, se utilizan `dataclasses` de Python, que permiten crear contenedores de estado claros y organizados. Esta es la "única fuente de verdad" de la aplicación, lo que evita que el estado se disperse y facilita la depuración.

4.1 Estructuras de Estado Centralizadas: `app_state.py`

Este archivo no contiene lógica, solo la definición de la estructura de los datos.

Dataclasses de Estado

Las siguientes clases son contenedores de datos. Su única función es almacenar y organizar el estado de la aplicación de una manera limpia y predecible.

PlotOptionsState

Almacena las opciones de visualización de los gráficos: qué espectros superponer, si la escala es logarítmica, si se muestran marcadores y el tipo de gráfico actual.

MeasurementDataState

Contiene los datos de la sesión actual: las lecturas de referencia (I_0) y de muestra (I), y si la sustracción de blanco está activa.

CalibrationState

Almacena todos los parámetros y modelos de calibración: la curva de sesión activa, los modelos de transferencia y los parámetros de calibraciones de referencia externas.

MeasurementControlState

Gestiona el estado de una medición en curso, especialmente en el modo secuencial: lleva la cuenta del progreso, acumula las muestras para promediar y guarda el nombre del archivo de salida.

La Vista: Interfaz de Usuario

La Vista comprende todos los componentes visuales con los que interactúa el usuario. Está diseñada de forma modular para promover la reutilización de código y la separación de responsabilidades.

5.1 El Panel Base: `base_panel.py`

Clase `BasePanel(QWidget)`

Esta clase base abstracta fue creada para que todos los paneles principales (pestañas) heredaran de ella. De esta forma, se asegura que todos tengan una estructura y una forma de comunicarse consistentes, evitando duplicar código.

`__init__(parent_app)`

Inicializa el panel y guarda una referencia a la aplicación principal para poder comunicarse con ella.

`_setup_base_ui()`

Configura la disposición visual básica, que es la misma para todos los paneles: una columna para gráficos y otra para controles.

`_init_plots()`

Método abstracto que las clases hijas deben implementar para inicializar sus gráficos específicos.

`_init_controls()`

Método abstracto que las clases hijas deben implementar para inicializar

sus controles específicos.

- `_connect_panel_signals()`
Conecta las señales internas del panel.
- `_log(message, ...)`
Permite que cualquier panel envíe mensajes al log centralizado de la aplicación.
- `_get_plot_colors()`
Devuelve la paleta de colores de gráfico correspondiente al tema visual actual.
- `on_theme_changed(is_dark_theme)`
Se activa para aplicar los colores del nuevo tema a los gráficos del panel.

5.2 Paneles de Pestañas (Implementaciones de BasePanel)

Panel de Espectro Principal: `main_spectrum_panel.py`



Clase `MainSpectrumPanel(BasePanel)`

Esta es la pestaña principal para la visualización de datos. Su trabajo es recibir los datos del controlador y renderizar los espectros de diferentes formas.

- `update_plot(plot_data)`
Es el método principal del panel. Recibe un diccionario con el estado actual y redibuja completamente el gráfico de Matplotlib.
- `_transform_data_for_plotting(intensity_data, ...)`
Convierte un espectro de intensidad a absorbancia o transmitancia.
- `_get_adjusted_absorbance(plot_data)`
Calcula el espectro de absorbancia ^ajustada aplicando un modelo de transferencia.
- `_finalize_plot_style(plot_data, ...)`
Aplica los toques finales al gráfico: etiquetas, título, leyendas, etc.
- `_on_plot_hover(event)`
Muestra una etiqueta emergente con el valor exacto de un punto de datos cuando el usuario pasa el ratón sobre él.
- `_on_superimpose_item_changed()`
Emite una señal cada vez que el usuario cambia la selección de espectros a superponer.

```
update_superimpose_list(measurement_data_raw,  
...)  Carga la lista de mediciones de la sesión para que el usuario pueda seleccionar  
       cuáles superponer.  
set_plot_options_state(show_markers,  
...)  Actualiza el estado de los checkboxes de opciones del gráfico para que  
       coincida con el estado global.
```

Panel de Curva de Concentración: `concentration_curve_panel.py`



Clase `ConcentrationCurvePanel(BasePanel)`

Esta pestaña es una herramienta flexible para que el usuario pueda construir gráficos personalizados de una variable X contra un valor Y medido.

```
request_direct_value_measurement()  
    Pide a la aplicación principal que realice una medición única y directa de  
    un valor específico.  
handle_direct_measurement_result(...)  Recibe el resultado de la medición directa y lo carga en la UI.  
add_data_point()  
    Añade un nuevo punto (X, Y) a la lista de datos del panel y actualiza la  
    tabla y el gráfico.  
remove_selected_data_point()  
    Elimina el punto que el usuario haya seleccionado en la tabla.  
update_table_and_plot()  
    Actualiza la tabla y el gráfico para reflejar los datos actuales.  
clear_points_for_current_view()  
    Limpia solo los puntos de la vista actual (sin afectar datos de otras longi-  
    tudes de onda).  
clear_all_points()  
    Borra todos los puntos de datos almacenados en este panel.
```

Panel de Calibración: `calibration_curve_tab.py`



Clase `CalibrationCurveTab(BasePanel)`

Esta es una de las partes más potentes de la aplicación, donde se crean los modelos matemáticos para la cuantificación precisa de muestras.

`request_direct_absorbance_measurement()`

Pide a la app principal una medición directa de absorbancia para un estándar de calibración.

`handle_direct_measurement_result(...)`

Procesa el resultado de la medición y lo carga en el campo correspondiente.

`add_calibration_point()`

Añade un nuevo punto de calibración a la lista.

`update_table_and_plot_all()`

Actualiza la tabla de puntos, recalcula los ajustes de regresión lineal y redibuja el gráfico.

`_calculate_fits(points_data)`

Es el núcleo matemático del panel. Usa SciPy para calcular las regresiones lineales de la calibración de sesión y del modelo de transferencia.

`_plot_abs_vs_conc(points_data)`

Dibuja el gráfico de la calibración de sesión.

`_plot_transfer_model(points_data)`

Dibuja el gráfico del modelo de transferencia.

`apply_calibration_to_main_app()`

Envía los modelos calculados a la aplicación principal para que puedan ser usados en la cuantificación.

`save_calibration_data_to_file()`

Guarda los puntos de calibración de la longitud de onda actual en un archivo CSV.

`load_calibration_data_from_file()`

Carga puntos de calibración desde un archivo CSV.

5.3 Diálogos y Widgets de Utilidad

Ventana de Ayuda: `help_dialog.py`



Clase `HelpDialog(QDialog)`

Implementa una ventana de ayuda completa con un árbol de temas navegable.

`_create_help_topics()`

Define un diccionario con todo el texto de ayuda formateado en HTML.

`_populate_tree(...)`

Construye el árbol visual a partir del diccionario de temas.

`_display_topic(item)`

Muestra el contenido HTML del tema seleccionado por el usuario.

Widgets de Utilidad: `utils.py`



Clase `PlotWidget(QWidget)`

Esta clase encapsula un gráfico de Matplotlib para poder integrarlo fácilmente en la interfaz de PyQt5 y controlar su estilo.

`apply_theme(theme_colors)`

Aplica una paleta de colores a todos los elementos del gráfico, permitiendo el cambio de tema.

`style_legend(theme_colors)`

Aplica el estilo del tema a la leyenda del gráfico.

`clear()`

Limpia el contenido del gráfico.

`draw()`

Redibuja el gráfico para que los cambios sean visibles.



Clase `CollapsibleGroupBox(QGroupBox)`

Este widget fue diseñado para organizar la interfaz. Se comporta como un grupo de controles normal, pero con un botón que permite al usuario mostrar u ocultar su contenido, reduciendo el desorden visual.

`_toggle_content()`

Muestra u oculta el contenido del grupo y actualiza el icono del botón.

6

Módulos de Soporte y Constantes

Estos módulos no contienen lógica compleja, pero son fundamentales para la consistencia y el mantenimiento del código.

6.1 Constantes Globales: `constants.py`

Este archivo centraliza todos los valores fijos. Esta práctica es crucial para el mantenimiento: si se necesita cambiar un color o un texto, solo se debe modificar en este archivo en lugar de buscarlo en todo el proyecto.

- **Parámetros del Sensor:** Las longitudes de onda del sensor y sus etiquetas para la UI.
- **Paletas de Colores:** Dos diccionarios que definen los colores para los gráficos en los temas oscuro y claro.
- **Hojas de Estilo (QSS):** Dos grandes strings que contienen el código CSS que le da estilo a toda la interfaz gráfica.
- **Textos de Ayuda:** Un diccionario que contiene todos los textos para los diálogos de ayuda contextual.

6.2 Tipos y Enumeraciones: `measurement_type.py`

Este archivo usa enumeraciones de Python para definir constantes con nombre. Esto mejora la legibilidad y previene errores de tipeo.

- **Clase 'MeasurementType':** Define los tipos de medición: CONTINUOUS, SINGLE_AVERAGED, SEQUENTIAL.
- **Enum 'PlotType':** Define los tipos de gráficos: INTENSITY, ABSORBANCE, TRANSMITTANCE.
- **Enum 'AppState':** Define los estados principales de la aplicación: IDLE, MEASURING_MAIN, MEASURING_DIRECT, CLOSING.